Response to Reviewer Comments (Round 2)

Contents

1 Response to Reviewer #2 **2**

2 Response to Reviewer #3 **2**

2.1 Comment 3.1 — Agent Abbreviations Omitted 2
2.2 Comment 3.2 — Variable x_{ij} Lacks Explicit Definition 3
2.3 Comment 3.3 — Duplicated "Appendix" Term 3
2.4 Comment 3.4 — SAR Variables Undefined 3
2.5 Comment 3.5 — SAR Definition and Illustrative Example 4
2.6 Comment 3.6 — Structure Notation Missing n_{bin} 5

3 Response to Reviewer #4 **5**

3.1 Comment 4.1 — Algebraic Modeling Languages Characterization 5
3.2 Comment 4.2 — Single-Agent Bottlenecks Not Detailed Before Introducing MAS 6
3.3 Comment 4.3 — Abrupt Narrative Shift to ECR 7
3.4 Comment 4.4 — Overlapping Contributions 8
3.5 Comment 4.5 — Agent Operational Boundaries and Message Protocols 8
3.6 Comments 4.6–4.12, 4.17–4.24 — Suggested References 9
3.7 Comment 4.13 — State-Dependent Decision Function Parameters 11
3.8 Comment 4.14 — Sub-Sequence Rule Structural Necessity 11
3.9 Comment 4.15 — LIFO Backtracking and Combined Failures 12
3.10 Comment 4.16 — Prompt Template Schema Drift Prevention 13
3.11 Comment 4.25 — Formal Computational Complexity Study 14
3.12 Comment 4.26 — Prompting Token Overhead Across Paradigms 15
3.13 Comment 4.27 — Structural Diversity and Solver Execution 16
3.14 Comment 4.28 — Cost Reduction Decomposition 17
3.15 Comment 4.29 — Human Expert as Core Architectural Layer 17

1 Response to Reviewer #2

We sincerely thank Reviewer #2 for the positive evaluation and the recommendation for acceptance. We are particularly grateful for the reviewer's insightful and constructive comments in the previous round. Addressing these comments has significantly strengthened our manuscript and deepened our understanding of the underlying issues. We truly appreciate the time and expertise invested, which have guided us to make substantial improvements.

2 Response to Reviewer #3

We sincerely thank Reviewer #3 for the positive evaluation and the recommendation for acceptance subject to minor revisions. The six comments on agent naming consistency, variable definitions, and the SAR formulation have helped us further tighten the notation and presentation. We are grateful for the careful reading and constructive suggestions that guided these refinements.

2.1 Comment 3.1 — Agent Abbreviations Omitted

2.1.1 Comment

"In lines 272-279, the redefinition and notation of the agents have resulted in the omission of their abbreviations (e.g., PM-agent, previously denoted as Project Manager Agent (currently denoted as a_{PM})), which are referenced in subsequent content."

2.1.2 Response

We are grateful to the reviewer for flagging this inconsistency; a uniform convention matters for readability, and we are glad to fix it. In the revised manuscript, we have unified the agent naming convention throughout the paper. The abbreviated forms (e.g., PM-Agent, DE-Agent) have been replaced by the full descriptive names accompanied by their symbolic notation (e.g., "the Project Manager Agent (a_{PM})"). In purely mathematical contexts such as set definitions and temperature parameters, the symbolic notation alone is retained. This eliminates the redundancy between the three naming layers and ensures consistent notation across all sections.

2.1.3 Modifications

1. **Section 3.1 (Agent Definition) — MODIFY**

   Before: "**PM-Agent:** Responsible for executing state transitions…", "**DE-Agent:** Its role is data engineering…", etc.

   After: "**Project Manager Agent ($a_{\mathbf{PM}}$):** Responsible for executing state transitions…", "**Data Engineer Agent ($a_{\mathbf{DE}}$):** Its role is data engineering…", etc.

2. **Sections 3.3–5.3 and Section 6 — MODIFY**

   Before: "the **PM-Agent** selects the next agent…"

   After: "the Project Manager Agent (a_{PM}) selects the next agent…"

   (The same replacement is applied to every agent mention across Sections 3.3–5.3 and Section 6; in purely mathematical contexts such as set definitions and temperature parameters, the symbolic notation alone is retained.)

2.2 Comment 3.2 — Variable x_{ij} Lacks Explicit Definition

2.2.1 Comment

"In line 486, while the definition of the variable x_{ij} may be inferred by readers with a background in operations research, it is advisable to provide an explicit definition to prevent ambiguity."

2.2.2 Response

We thank the reviewer for this careful reading; an undefined symbol is a genuine distraction for the reader, and we are grateful it was caught. In the revised manuscript, we have added an explicit definition of x_{ij} immediately after its first contextual mention in the illustrative example, clarifying that it denotes the number of empty containers repositioned from port i to port j.

2.2.3 Modifications

1. **Section 3.6 (Illustrative Example) — MODIFY**

   Before: "...the unit transportation costs c_{ij} range from 2 to 10 ¥/TEU."

   After: "...the unit transportation costs c_{ij} range from 2 to 10 ¥/TEU, and x_{ij} denotes the number of empty containers repositioned from port i to port j."

2.3 Comment 3.3 — Duplicated "Appendix" Term

2.3.1 Comment

"In line 789, the term 'Appendix' is repeated within the sentence."

2.3.2 Response

We thank the reviewer for this good catch; the duplicated prefix is an artifact of the `elsarticle` class, and we are glad it was flagged. The duplication arose because the `\ref` command in the `elsarticle` class already prepends "Appendix" to appendix section numbers. Writing `Appendix~\ref{app2}` therefore rendered as "Appendix Appendix B." In the revised manuscript, the redundant prefix has been removed.

2.3.3 Modifications

1. **Section 5.1 (Case Setup) — MODIFY**

   Before: "...are reported in Appendix Appendix B."

   After: "...are reported in Appendix B."

2. **Section 5.2 (Problem Description) — MODIFY**

   Before: "The complete mathematical formulation is provided in Appendix Appendix C."

   After: "The complete mathematical formulation is provided in Appendix C."

2.4 Comment 3.4 — SAR Variables Undefined

2.4.1 Comment

"In lines 826-828, the definition of the Structural-Agreement Rate (SAR) is presented with the formulation structure $(n_{vars}, n_{constrs}, n_{bin}, n_{int}, nnz)$; however, these variables lack definition."

2.4.2 Response

We appreciate the reviewer's close attention to detail here; leaving the tuple components implicit was an oversight that made the definition less precise. In the revised manuscript, we have added explicit definitions for all five components of the formulation structure tuple immediately after the SAR definition.

2.4.3 Modifications

1. **Section 5.4 (Formulation Stability Analysis) — MODIFY**

   Before: "...the Structural-Agreement Rate (SAR), defined as pairwise agreement of the formulation structure $(n_{\text{vars}}, n_{\text{constrs}}, n_{\text{bin}}, n_{\text{int}}, \text{nnz})$, has limited discriminative power."

   After: "...the Structural-Agreement Rate (SAR), defined as pairwise agreement of the formulation structure $(n_{\text{vars}}, n_{\text{constrs}}, n_{\text{bin}}, n_{\text{int}}, \text{nnz})$, has limited discriminative power. Here, n_{vars} denotes the total number of decision variables, n_{constrs} the number of constraints, n_{bin} the number of binary variables, n_{int} the number of integer variables, and nnz the number of non-zero coefficients in the constraint matrix."

2.5 Comment 3.5 — SAR Definition and Illustrative Example

2.5.1 Comment

"SAR is described as the 'pairwise agreement of the formulation structure,' which does not adequately convey the concept of 'Rate.' An example illustrating the calculation of SAR would be beneficial."

2.5.2 Response

We are grateful to the reviewer for this constructive suggestion; writing SAR as an equation makes the stability analysis more rigorous. In the revised manuscript, we have added a formal definition of SAR as an equation, specifying that it is computed as the ratio of matching formulation-structure pairs to all possible pairs among feasible runs. This makes the "Rate" interpretation explicit: SAR equals 1 when all feasible runs produce identical structures, and 0 when every run yields a distinct structure. The accompanying variable definitions (F_i, R', indicator function) further clarify the computation.

2.5.3 Modifications

1. **Section 5.4 (Formulation Stability Analysis) — ADD**

   After: The variable definitions (n_{vars}, n_{constrs}, n_{bin}, n_{int}, nnz), the following formal definition has been added:
   "Let $F_i = (n^i_{\text{vars}}, n^i_{\text{constrs}}, n^i_{\text{bin}}, n^i_{\text{int}}, \text{nnz}^i)$ denote the formulation structure observed in the i-th run that produced a valid formulation. The SAR is then computed as $\text{SAR} = \frac{\sum_{1 \le i < j \le R'} \mathbf{1}(F_i = F_j)}{\binom{R'}{2}}$, where R' is the number of runs that yielded a formulation structure and $\mathbf{1}(\cdot)$ is the indicator function."

2.6 Comment 3.6 — Structure Notation Missing n_{bin}

2.6.1 Comment

"Additionally, when structures are mentioned (e.g., line 843 (379,130,370,718)), the omission of n_{bin} leads to potential confusion."

2.6.2 Response

We thank the reviewer for raising this; a column that is silently omitted can easily look like an error, and the reviewer is right to ask about it. In the ECR formulation, all integer variables are binary ($n_{\text{bin}} = n_{\text{int}}$), so the original table omitted n_{bin} to avoid redundancy. In the revised manuscript, we have added an explicit note after the table stating this relationship, ensuring that readers are not confused by the apparent omission.

2.6.3 Modifications

1. **Section 5.4 (Formulation Stability Analysis) — ADD**

   After Table 4: The following note has been added: "Note that $n_{\text{bin}} = n_{\text{int}}$ for all formulations in Table 4, as every integer variable in the ECR model is binary; the column is therefore omitted to avoid redundancy."

3 Response to Reviewer #4

We sincerely thank Reviewer #4 for the thorough and constructive evaluation of our manuscript. The 29 detailed comments prompted substantial improvements, including new experimental analyses (token overhead across paradigms, solver-execution characterization), the computational complexity analysis, clarifying refinements to the SAR formula and the subsequence rule, a consolidated literature review incorporating four additional references, and multiple clarifications throughout. We are grateful for the time and expertise that made this round of revision significantly strengthen the paper.

3.1 Comment 4.1 — Algebraic Modeling Languages Characterization

3.1.1 Comment

"The characterization of algebraic modeling languages as inadequate overlooks their structural role as target compilers for the code generated by the system."

3.1.2 Response

We thank the reviewer for this fair point. The original wording did not adequately acknowledge the structural role of AMLs as intermediate representations and compilation targets. In the revised manuscript, we have softened the characterization in the Introduction and added an explicit statement in Section 2.2 recognizing that AMLs serve as solver-agnostic intermediate representations that can function as compilation targets for automatically generated code.

3.1.3 Modifications

1. **Section 1 (Introduction) — MODIFY**

   Before: "...they remain inadequate in realizing end-to-end optimization from unstructured business descriptions to MIP formulation."

   After: "...their reliance on expert-authored algebraic expressions means they

cannot bridge the gap between unstructured business descriptions and MIP formulation without human intervention."

2. **Section 2.2 (Traditional MIP Solution) — ADD**

   After: "...decoupling models from data and enhancing modeling efficiency."

   Added: "These languages serve as structured intermediate representations that formalize optimization models in a solver-agnostic manner, and modern systems such as SOP-MAS can leverage them as compilation targets for automatically generated code."

3.2 Comment 4.2 — Single-Agent Bottlenecks Not Detailed Before Introducing MAS

3.2.1 Comment

"The text introduces the concept of multi-agent collaboration before detailing the precise failures or bottlenecks of single-agent setups in operations research."

3.2.2 Response

We are grateful for this important point; spelling out the limitations of the single-agent paradigm is important for motivating the multi-agent design, and the reviewer is right to push us on this. In the revised manuscript, we have strengthened the treatment of single-agent limitations at two points. First, the Introduction now cites concrete failure modes of single-agent LLMs in OR tasks (e.g., omitted variables or constraints, logically incorrect formulations, compilable but infeasible code) before the MAS paradigm is introduced. Second, we retitle Section 2.3 as "Single-Agent LLM Paradigm in Scientific Computing and OR" to flag the single-agent identity at the section level, and make its characteristic bottlenecks (hallucination, opacity, and limited multi-step reasoning) explicit; the opening of Section 2.4 now attributes the multi-agent motivation directly to these documented single-agent limitations.

3.2.3 Modifications

1. **Section 1 (Introduction) — ADD**

   After: "...which considerably restricts their practical application in critical decision-making systems."

   Added: "Empirical studies have documented characteristic failure modes of single-agent LLMs in optimization tasks: omitting decision variables or constraints that are implicit in the problem description, generating syntactically valid but logically incorrect mathematical formulations, and producing solver code that compiles yet yields infeasible or suboptimal solutions. These errors are particularly difficult to detect in the absence of an independent verification mechanism, as the same agent that generates the model is also responsible for validating it."

2. **Section 2.3 (subsection title) — MODIFY**

   Before: "Emerging Paradigms from LLMs in Scientific Computing and OR"

   After: "Single-Agent LLM Paradigm in Scientific Computing and OR"

3. **Section 2.4 (opening sentence) — MODIFY**

Before: "In response to the constraints associated with LLMs, researchers..."

After: "In response to the constraints of *single-agent* LLMs documented above, researchers..."

3.3 Comment 4.3 — Abrupt Narrative Shift to ECR

3.3.1 Comment

"The narrative shifts abruptly from generalized large language model limitations to the highly specific application domain of empty container repositioning."

3.3.2 Response

We thank the reviewer for this good suggestion; opening with the motivating application rather than the methodology gives the paper a much clearer flow. In the revised manuscript, we have restructured the Introduction to lead with the ECR problem as the motivating application, establishing its characteristics and challenges before discussing LLM-based solutions. ECR is now introduced in the opening sentence, and the subsequent discussion of MIP limitations, LLM potential, and multi-agent collaboration is framed as responses to the specific demands of this problem rather than a generic narrative that abruptly shifts domain. To keep this ECR-first framing consistent across the paper, the Literature Review (Section 2) has been reorganized to open with the ECR challenges (Section 2.1) before the modeling paradigms are reviewed, and a two-part roadmap now signposts this progression.

3.3.3 Modifications

1. **Section 1 (Introduction, Paragraph 1) — MODIFY**

   Before: "Mixed-Integer Programming (MIP) constitutes a fundamental instrument within the domain of Container Supply Chain Management (SCM)..."

   After: "Empty Container Repositioning (ECR), the logistical challenge of repositioning surplus empty containers from locations with excess supply to locations with unmet demand, is a central problem in container supply chain management. Optimizing ECR requires multi-period, multi-node, and multi-modal transportation decisions under dynamic supply–demand imbalances, a task commonly formulated as a Mixed-Integer Program (MIP). In practice, however, translating such complex business scenarios into a correct MIP formulation is heavily dependent on domain experts..."

2. **Section 1 (Introduction, Paragraph 4) — MODIFY**

   Before: "Empty Container Repositioning (ECR) is one of the most representative and challenging problems in container SCM..."

   After: "We employ the ECR problem introduced above as an empirical testbed, as its multi-stakeholder, multi-modal complexity and the dual demand for mathematical rigor and business interpretability make it a compelling representative of the class of problems that the proposed framework is designed to address."

3. **Section 2.1 (Overview of Empty Container Repositioning Challenges) — REORDER**

   This subsection is relocated from the end of Section 2 (previously Section 2.4) to open the Literature Review, so that the ECR challenges establish the context before the modeling paradigms are reviewed. Its content is unchanged.

4. **Section 2 (Literature Review, opening paragraph) — ADD**

   After: "...employs container SCM as a representative application scenario."

   Added: "We organize this review in two parts. We first survey the empty container repositioning challenges that motivate this work. We then trace the evolution of MIP modeling paradigms used to address such problems, from the expert-driven traditional approach, through the emerging paradigm in which a *single* LLM handles the entire modeling-to-solving pipeline, to the multi-agent collaborative paradigm on which our work builds."

5. **Section 2.1 (end of ECR subsection) — ADD**

   After: "...their collaboration can effectively diminish the detention rates of empty containers."

   Added: "Resolving such challenges in practice requires casting them as solvable Mixed-Integer Programs; the paradigms for modeling and solving these programs are reviewed below."

3.4 Comment 4.4 — Overlapping Contributions

3.4.1 Comment

"The listed contributions overlap significantly, with the third and fourth items essentially restating the same empirical validation from different viewpoints."

3.4.2 Response

We thank the reviewer for this sharp observation. The reviewer rightly identified that the third and fourth contributions did overlap, and we agree they should be consolidated. We agree that the original third and fourth contributions described the same empirical work from different angles. In the revised manuscript, we have merged them into a single contribution that presents the framework implementation and its empirical validation on ECR as an integrated claim, and renumbered the contributions from fourfold to threefold.

3.4.3 Modifications

1. **Section 1 (Introduction) — MODIFY**

   Before: "...are fourfold: 1)...3) The implementation of an end-to-end automated modeling and solving framework...4) The exploratory application of LLM and MAS in container supply chain optimization..."

   After: "...are threefold: 1)...3) The implementation and empirical validation of an end-to-end framework that translates natural language business descriptions into optimized solutions, demonstrated through the sea-land coordinated ECR problem faced by shipping companies in the Liaoning coastal port cluster, which achieved a 12.18% reduction in total operational costs and thereby verifies both the method's effectiveness and its potential for the digital and intelligent transformation of the port and shipping industry."

3.5 Comment 4.5 — Agent Operational Boundaries and Message Protocols

3.5.1 Comment

"The roles of the six distinct agents are mapped out without explaining the exact operational boundaries or message protocols that govern their handoffs."

3.5.2 Response

We thank the reviewer for this helpful observation; separating three distinct concerns that we had packed into a single paragraph has made the design much clearer. In the revised manuscript, the operational boundaries, the message protocol, and the handoff mechanism are now treated as distinct concerns rather than a single paragraph. Section 3.1 states each agent's operational boundary (delineated by its prompt template $\mathcal{T}_a$). A new dedicated subsection, Section 3.2 (Inter-Agent Communication Protocol), formalizes the shared message pool C as a blackboard medium, specifies the read/append access primitives and the per-agent output formats (JSON parameters and model, executable code, diagnostic report, business report), and states the no-direct-invocation separation of concerns. The handoff, i.e., the order in which agents are activated, is governed by the sub-sequence rule presented in the workflow orchestration (Section 3.3).

3.5.3 Modifications

1. **Section 3.1 (Agent Definition) — ADD**

   After: The agent tuple definition ($a = (f_a, L_a, \mathcal{T}_a, t_a, \mathcal{H}_a)$).

   Added: "The operational boundary of each agent is delineated by its prompt template $\mathcal{T}_a$, which specifies the agent's permitted inputs, expected output format, and task scope, confining each agent to a well-defined sub-task."

2. **Section 3.2 (Inter-Agent Communication Protocol) — NEW SUBSECTION**

   After: "The six agents do not communicate peer-to-peer but coordinate exclusively through a shared message pool C, which serves as a blackboard-style communication medium. The pool C is an ordered log in which each entry records the producing agent together with its structured output. Two primitives govern access: on activation, an agent reads the current state of C as context, and after computing its result it appends a new entry to C. No agent directly invokes or modifies another agent's internal state, which enforces a strict separation of concerns and leaves every intermediate artifact available for inspection and verification." The subsection further specifies the per-agent output formats (structured JSON from the Data Engineer Agent, a markdown-formatted model from the Modeling Expert Agent, executable code from the Python Developer Agent, a diagnostic report from the Testing Engineer Agent, and a business report from the Business Analyst Agent) and states that the activation order follows the sub-sequence rule of Section 3.3.

3.6 Comments 4.6–4.12, 4.17–4.24 — Suggested References

3.6.1 Comments

The reviewer suggested fifteen additional references (Comments 4.6–4.12 and 4.17–4.24) for integration at various points of the manuscript.

3.6.2 Response

We sincerely thank the reviewer for these thoughtful suggestions and for the time invested in curating such a diverse reading list. We have carefully read and learned from each of the fifteen recommended works; collectively they broadened our perspective on the neighboring fields of LLM-driven optimization, Transformer-based decision-making, and supply-chain digitalization, and we are grateful for the reviewer's guidance in pointing us to them. When integrating

these works into the manuscript, we weighed each one against the focused scope of this paper, namely LLM-driven multi-agent collaboration for the automated modeling and solution of OR (MIP) problems, instantiated on empty container repositioning (ECR). Four works that speak directly to this scope have been incorporated into the revised Literature Review (Section 2); the remaining eleven, while valuable in their own domains, address topics somewhat beyond the present scope and are therefore not incorporated here, though several offer useful pointers for our future extensions. The per-reference disposition and rationale are documented in Table 1.

The four incorporated references strengthen the review as follows. Zhang et al. (2025, *Computers & OR*) exemplifies the shift from purely expert-built models toward knowledge- and data-driven hybrid optimization, and is added in Section 2.2. Yuan et al. (2024, *FITEE*) and Li et al. (2026, *CAAI Trans. Intell. Technol.*) broaden the coverage of LLM/Transformer-based decision-making and optimization, and are added in Section 2.3. Hu et al. (2026, *BPMJ*) contextualizes the supply-chain-digitalization background of ECR, and is added in Section 2.1. Where a reviewer-suggested section did not coincide with the revised structure, we placed the citation at its most natural location.

Table 1: Disposition of the fifteen suggested references (Comments 4.6–4.12, 4.17–4.24).

#	Reference (topic)	Disposition
4.6	Guo et al. (2024)	Not incorporated: planetary-rover escape-path planning, outside OR modeling
4.7	Zhao et al. (2024)	Not incorporated: target-driven visual navigation for vehicles
4.8	Huang et al. (2025)	Not incorporated: LLM for cold-start recommendation, not optimization
4.9	Zhang et al. (2025, IJPR)	Not incorporated: SMT scheduling metaheuristics, without an LLM/MAS component
4.10	**Zhang et al. (2025, COR)**	**Incorporated, Sec. 2.2: knowledge-/data-driven hybrid optimization**
4.11	Chen et al. (2026)	Not incorporated: causal multi-modal root-cause analysis
4.12	Xing et al. (2025)	Not incorporated: multimodal AI for cultural-heritage sculpture analysis
4.17	Long et al. (2026)	Not incorporated: AutoML scheduling for space-surveillance
4.18	**Yuan et al. (2024)**	**Incorporated, Sec. 2.3: Transformer+RL decision-making survey**
4.19	Liu et al. (2025)	Not incorporated: deep-RL sample-efficient method, outside the LLM-MAS paradigm
4.20	**Li et al. (2026)**	**Incorporated, Sec. 2.3: LLM fine-tuning for combinatorial optimization**
4.21	**Hu et al. (2026)**	**Incorporated, Sec. 2.1: supply-chain digitalization context**
4.22	Qin et al. (2026)	Not incorporated: software fault localization (semantic code search)
4.23	Huang et al. (2026)	Not incorporated: LLM for mobile-app usage data generation
4.24	Yue et al. (2026)	Not incorporated: ADS-B aviation data authentication, outside the ECR/MAS scope

3.6.3 Modifications

The four incorporated references are introduced at three points in Section 2 (Yuan et al., 2024 and Li et al., 2026 appear together in a single sentence in Section 2.3):

1. **Section 2.1 (Overview of ECR Challenges) — ADD (Comment 4.21)**

After: "The broader trend of supply chain digitalization (Hu et al., 2026) further accentuates the need for agile, data-driven repositioning decisions."

2. **Section 2.2 (Traditional MIP Solution) — ADD (Comment 4.10)**

   After: "More recently, knowledge- and data-driven hybrid methods have begun to automate scheduling and optimization beyond purely expert-built models (Zhang et al., 2025)."

3. **Section 2.3 (Single-Agent LLM Paradigm) — ADD (Comments 4.18, 4.20)**

   After: "Parallel efforts explore the integration of Transformer architectures with reinforcement learning for general decision-making (Yuan et al., 2024) and the fine-tuning of LLMs for combinatorial optimization (Li et al., 2026), further expanding the application frontier of LLM-driven optimization."

3.7 Comment 4.13 — State-Dependent Decision Function Parameters

3.7.1 Comment

"The mathematical formalization uses a state-dependent decision function but omits the exact parameters that define the current contextual state dictionary."

3.7.2 Response

We appreciate the reviewer's attention to detail here; spelling out each component of the state tuple removes a real ambiguity. The reviewer is right that the original phrasing listed the components of the state tuple only in running text, without explicitly tying each symbol to its meaning. In the revised manuscript we have rewritten the definition of $s_k = (P, C, K - k, \mathcal{A}_k)$ (Section 3.3, Step 2) so that each component is now stated explicitly: P denotes the problem description, C the message pool history, $K - k$ the remaining selection quota, and $\mathcal{A}_k$ the current selectable agent set. Together these four components fully characterize the contextual state available to the a_{PM} selection function π_{PM}.

3.7.3 Modifications

1. **Section 3.3 (Step 2, Agent Selection) — MODIFY**

   Before: "... where s_k denotes the current state comprising the problem description, message pool history, remaining selection quota, and the current selectable set $\mathcal{A}_k$..."

   After: "... where s_k is the current state, with P denoting the problem description, C the message pool history, $K - k$ the remaining selection quota, and $\mathcal{A}_k$ the current selectable agent set..."

3.8 Comment 4.14 — Sub-Sequence Rule Structural Necessity

3.8.1 Comment

"The sub-sequence rule relies on an ordered tuple sequence, yet the text fails to justify the structural necessity of this rigid directional hierarchy."

3.8.2 Response

We thank the reviewer for prompting this clarification; the rationale behind the directional hierarchy is important to the design, and stating it explicitly makes the paper clearer. The directional hierarchy is not an arbitrary design choice but reflects the intrinsic dependency chain of the optimization modeling process: data extraction (a_{DE}) must precede model formulation (a_{ME}), which must precede code generation (a_{PY}), whose output is validated by the Testing Engineer (a_{TE}), with the Business Analyst (a_{BA}) finally interpreting the verified solution; each stage's output is the next stage's prerequisite. The ordered tuple $\mathcal{A}^{\uparrow}$ is therefore precisely the topological order of this linear dependency chain. We have added a justification sentence in Section 3.3 to clarify this structural rationale.

3.8.3 Modifications

1. **Section 3.3 (introductory paragraph) — ADD the sub-sequence rule and its structural rationale**

   Before: "This framework supports task execution through structured collaboration among specialized agents. The sequence of agent interactions under SOP is illustrated in Figure 1 and comprises the following steps:"

   After: "...illustrated in Figure 1. Central to this orchestration is the *sub-sequence rule* (the order-constraint component of the SOP governance): agents are activated in the fixed role order $a_{\mathrm{DE}} \to a_{\mathrm{ME}} \to a_{\mathrm{PY}} \to a_{\mathrm{TE}} \to a_{\mathrm{BA}}$, and once selected, only downstream roles remain selectable. This order traces the linear dependency chain of MIP modeling, in which each agent's output is the next one's input (parameters → model → code → validation → interpretation), so out-of-order selection would break the chain. The workflow comprises the following steps:"

3.9 Comment 4.15 — LIFO Backtracking and Combined Failures

3.9.1 Comment

"The exception backtracking mechanism assumes a last-in-first-out stack model without explaining how the framework handles cases where errors originate from a combined failure of multiple previous agents."

3.9.2 Response

We thank the reviewer for raising this point; combined, multi-agent failures are exactly the case the backtracking mechanism is designed to handle. The LIFO backtracking mechanism handles combined failures through sequential agent querying. When the stack is popped in reverse order, each agent independently evaluates whether its own output contributed to the failure. In a combined failure scenario, multiple agents may sequentially claim responsibility ($\delta_k = 1$): the first agent (nearest to the failure point) corrects its output and the framework resumes forward modeling; if the error persists, the next backtracking round reaches the next responsible agent. This iterative process continues until all contributing errors are resolved or the collaboration budget K is exhausted. The mechanism is illustrated in the illustrative example (Section 3.6), where a_{PY} returns $\delta_3 = 0$ (not responsible) and a_{ME} returns $\delta_2 = 1$ (responsible), demonstrating the sequential propagation logic. To make this explicit in the manuscript, we have added a sentence in Section 3.3 stating that combined failures are resolved over successive backtracking rounds.

3.9.3 Modifications

1. **Section 3.3 (Step 7, Exception Backtracking) — ADD**

   After: "...continues the LIFO traversal until an agent claims responsibility or the stack is exhausted. In particular, when a failure originates from multiple agents, the framework resolves them over successive backtracking rounds: each round in which some agent claims responsibility ($\delta_k = 1$) produces a revised output and a fresh forward pass, and any error that remains triggers the next round, popping the next agent in reverse order, until all culprits are corrected or the collaboration budget K is exhausted."

3.10 Comment 4.16 — Prompt Template Schema Drift Prevention

3.10.1 Comment

"The prompt template for the modeling expert agent restricts the generation scope to markdown formatting but lacks programmatic constraints to prevent structural schema drift."

3.10.2 Response

We thank the reviewer for this precise observation, which exposed an inconsistency between our manuscript's claims and the actual implementation. We verified the codebase and confirm the reviewer's assessment: the Modeling Expert Agent's (a_{ME}) prompt specifies a markdown schema (headings `# parameter`, `# variable`, `# objective`, `# constraint`) but imposes no programmatic schema gate at the template level. We have corrected two places in the manuscript (Sections 3.2 and 3.5) that had over-stated the enforcement level, and we clarify the actual mechanism here.

The a_{ME} specification is consumed by the downstream Python Developer Agent (a_{PY}) as structured natural-language context within the shared message pool, not by a deterministic parser; structural consistency of the markdown is therefore governed by instruction-level specification together with downstream behavioral feedback, rather than by a hard schema contract. This is a deliberate trade-off rather than an oversight: a mathematical optimization model involves symbolic notation (indexed sets, summations, quantifiers, and comparison operators) that does not map cleanly onto a rigid JSON schema, whereas markdown preserves the expressive fluency such notation requires while supplying lightweight structural cues for downstream interpretation. The same consideration explains the format asymmetry across agents: the Data Engineer Agent (a_{DE}) emits JSON because its output (a parameter table) is naturally key-value structured, whereas a_{ME}'s mathematical model is not.

Although no proactive schema gate is imposed on the a_{ME} output, the framework provides a reactive safeguard that prevents schema drift from propagating silently: any structurally degraded specification that leads a_{PY} to generate non-executable or infeasible code triggers the exception-backtracking mechanism (Section 3.3), which can localize the fault to a_{ME} and request a refined specification; and because every terminal output must additionally pass solver execution and objective-value verification, a schema defect cannot yield a silently incorrect published result. We concur with the reviewer that a programmatic prevention layer is a worthwhile complement, and note that constrained-decoding techniques (e.g., grammar-constrained sampling) could enforce the markdown heading schema at generation time; we identify this integration as a direction for future hardening.

3.10.3 Modifications

1. **Section 3.2 (Inter-Agent Communication Protocol) — MODIFY**

Before: "The Data Engineer Agent (a_{DE}) and Modeling Expert Agent (a_{ME}) emit structured JSON specifying the extracted parameters and the mathematical model"

After: "The Data Engineer Agent (a_{DE}) emits structured JSON specifying the extracted parameters, and the Modeling Expert Agent (a_{ME}) emits a markdown-formatted model specification (parameters, variables, objective, constraints)"

2. **Section 3.5 (Mechanisms for Hallucination Mitigation) — MODIFY** (two overstatements of JSON enforcement)

   Before (item 1): "... the LLMs are constrained to produce output strictly in JSON format or in accordance with specific code templates ..."

   After (item 1): "... the LLMs are constrained to produce output in role-specific structured formats (JSON for parameter and diagnostic outputs, a markdown model specification, or executable code templates) ..."

   Before (item 2): "Utilizing JSON as the communication protocol between agents inherently restrict each agent's latitude for unstructured 'free-form' output ..."

   After (item 2): "The use of role-specific structured formats (JSON and markdown specifications) as the inter-agent communication protocol restricts each agent's latitude for unstructured 'free-form' output ..."

3.11 Comment 4.25 — Formal Computational Complexity Study

3.11.1 Comment

"The efficiency analysis asserts that runtime is scale-independent based on text length without providing a formal computational complexity study of the local orchestration layers."

3.11.2 Response

We sincerely appreciate this thoughtful observation; a formal complexity analysis is something the manuscript owed its readers, and the reviewer is right to insist on it. We have expanded the complexity treatment into a dedicated subsection, Section 3.4 (Computational Complexity Analysis), that provides a formal study of the orchestration layer. The end-to-end runtime is decomposed into three components, $T_{\text{total}} = T_{\text{orch}} + T_{\text{LLM}} + T_{\text{solver}}$, and the orchestration overhead is derived as $T_{\text{orch}} = O(K \cdot |\mathcal{A}|)$, which for the fixed agent set ($|\mathcal{A}| = 6$) and a small collaboration budget ($K \leq 5$) reduces to a scale-independent constant. The scale-dependent costs, LLM inference and solver execution, are shown to reside outside the orchestration layer; the former is empirically confirmed as the dominant term in the companion efficiency study (Section 4.4).

3.11.3 Modifications

1. **Section 3.4 (Computational Complexity Analysis) — ADD**

   After: "The end-to-end cost is decomposed as $T_{\text{total}} = T_{\text{orch}} + T_{\text{LLM}} + T_{\text{solver}}$. The orchestration overhead is derived as $T_{\text{orch}} = O(K \cdot |\mathcal{A}|)$ from the message-pool append ($O(K)$), selectable-set construction ($O(K \cdot |\mathcal{A}|)$), and stack ($O(K)$) operations, which reduces to a scale-independent constant for the fixed agent set ($|\mathcal{A}| = 6$) and small K. The LLM-inference (T_{LLM}) and solver-execution (T_{solver}) terms are identified as the scale-dependent components external to the orchestration layer, with the former empirically confirmed as the dominant term

in Section 4.4. In summary, $T_{\text{orch}} = O(K \cdot |\mathcal{A}|)$ reduces to $O(1)$ with respect to the MIP problem scale, and the end-to-end runtime is governed by T_{LLM} rather than by orchestration."

3.12 Comment 4.26 — Prompting Token Overhead Across Paradigms

3.12.1 Comment

"The baseline comparison controls the language model backbone but fails to account for variations in prompting token overhead across different paradigms."

3.12.2 Response

We thank the reviewer for this important methodological observation; fairness of comparison extends well beyond the model backbone, and the reviewer is entirely right that the prompting-overhead dimension should be reported explicitly. We agree that controlling the language-model backbone is necessary but not sufficient for a fair comparison: the prompting token overhead incurred by each paradigm is an equally important dimension that warrants explicit reporting.

To address this, we conducted a supplementary token-overhead analysis, reported in the revised Section 4.4 (new Table 3 and Figure 5). We report the mean number of LLM calls and the mean total tokens consumed per instance.

The results show that prompting overhead scales monotonically with the complexity of collaboration: SPM (1 call, ≈1.8k tokens), CoT (3 calls, ≈4.9k), OptiMUS (≈7 calls, ≈9.8k), CoE (≈11 calls, ≈23k), and SOP-MAS (≈11 calls, ≈30k). SOP-MAS incurs the highest prompting overhead, which is the expected cost of its multi-agent collaboration with exception backtracking.

Figure 5(b) shows a positive association between mean total tokens and AR: SOP-MAS consumes the largest prompting budget (≈30k tokens per instance) and achieves the highest accuracy in Table 1 (86.59% versus 48.08–77.35% for the baselines). In typical ECR applications, per-instance repositioning, leasing, and holding costs exceed API expenditure by orders of magnitude.

3.12.3 Modifications

1. **Section 4.4 (Computational Efficiency Analysis) — ADD token-overhead analysis**

   A new paragraph, Table 3 (mean LLM calls and total tokens per paradigm), and Figure 5 (two panels) are appended to Section 4.4. The added text reads: "We also report prompting token consumption for each paradigm. Table 3 and Figure 5 list the mean number of LLM calls and mean total tokens per instance, together with the AR reported in Table 1. Prompting overhead increases monotonically with collaboration complexity, from single-shot SPM (≈1.8k tokens) through CoT, OptiMUS, and CoE to SOP-MAS (≈30k tokens), consistent with multi-agent collaboration and exception backtracking. Figure 5(b) shows a positive association between mean total tokens and AR: SOP-MAS consumes the largest prompting budget (≈30k tokens per instance) and achieves the highest accuracy in Table 1 (86.59% versus 48.08–77.35%). In typical ECR applications, per-instance repositioning, leasing, and holding costs exceed API expenditure by orders of magnitude."

3.13 Comment 4.27 — Structural Diversity and Solver Execution

3.13.1 Comment

"The structural diversity observed in the stability experiment is dismissed as equivalent encoding, neglecting the potential impact on solver execution variations."

3.13.2 Response

We thank the reviewer for this important clarification request, and we agree that mathematical equivalence of formulations does not imply identical solver execution. Structurally distinct encodings differ in size, sparsity, and conditioning, and can therefore differ in solve time, presolve behavior, and numerical characteristics. Our use of "equivalent encodings" in Section 5.4 denotes alternative representations of the same logical constraints (hence the same decision problem), not identity of solver behavior. Agreement on the optimal objective is a consequence of this encoding equivalence, not its definition; we also characterize the solver-execution dimension explicitly below.

We re-executed the solver on each of the distinct formulation structures produced in the stability experiment (Section 5.4), re-using the generated model code. The table below reports, for each feasible structure, the formulation size, the mean wall-clock solve time (5 repeats), the simplex iteration count, and the resulting objective.

$(n_{\text{vars}}, n_{\text{constrs}}, \text{nnz})$	Type	Wall (ms)	Simplex iters	Objective
$(361, 65, 644)$	compact	2.4	107	1,355,388
$(361, 130, 709)$	compact	5.4	107	1,355,388
$(370, 65, 653)$	compact	2.7	107	1,355,388
$(370, 74, 662)$	compact	2.2	107	1,355,388
$(370, 130, 718)$	compact	2.3	107	1,355,388
$(458, 221, 884)$	compact	5.2	102	1,355,388
$(975, 130, 1798)$	expanded	5.6	107	1,355,388
$(975, 984, 2652)$	expanded	8.0	85	1,355,388

Across the eight feasible structures (n_{vars} from 361 to 975; nnz from 644 to 2,652), every encoding converges to the identical optimum (CNY 1,355,388) at the root node. Solver execution does vary with structure, as the reviewer anticipates: simplex iterations range from 85 to 107, and wall-clock solve time from $\approx$2 ms for the compact encodings ($n_{\text{vars}} \approx 361$–$458$) to $\approx$6–8 ms for the expanded encodings ($n_{\text{vars}} \approx 975$), a roughly 2–5$\times$ difference. (Two further generated structures recorded no constraints and did not yield a feasible solve; they are excluded here and correspond to the code-execution failure mode already discussed in Section 5.4.)

Crucially, this variation is bounded and inconsequential for the framework: every feasible structure attains the correct optimum, and the entire solve-time range (< 11 ms) is negligible relative to the LLM-driven modeling latency ($\approx$48 s per instance, Section 4.4). We therefore clarify that "equivalent encoding" means alternative representations of the same logical constraints; the resulting solver-execution variation is real but bounded, and does not affect the framework's correctness or its end-to-end runtime. We have added a clarifying sentence in Section 5.4 accordingly, with the execution dimension characterized in this response.

3.13.3 Modifications

1. **Section 5.4 (Formulation Stability Analysis) — ADD clarifying sentence.**

   > **After:** "... the remaining runs adopt alternative but mathematically equivalent encodings (e.g., variable upper bounds instead of explicit capacity constraints, or time-window pre-filtering of repositioning arcs)."

Added: "Here, 'equivalent' means that the encodings represent the same logical constraints via different formulations (e.g., capacity as an explicit inequality versus a variable upper bound), and therefore induce the same optimum; it does not claim identical solver behavior. The structurally distinct encodings do differ in solve time and simplex iteration count, but these differences are bounded and negligible at this scale."

3.14 Comment 4.28 — Cost Reduction Decomposition

3.14.1 Comment

"The reported reduction in total operational cost combines disparate logistics activities, making it difficult to isolate the exact efficiency gains of lateral land routes."

3.14.2 Response

We thank the reviewer for this thoughtful point; isolating the marginal effect of the lateral land route is essential for interpreting the cost reduction, and we agree the original text did not make this clear. The comparison between Scenario 1 (hinterland lateral repositioning enabled) and Scenario 2 (inter-dry-port exchange disabled) is a controlled pairwise contrast: the two scenarios differ in exactly one design variable (the availability of lateral land routes) while sharing the same network structure, cost parameters, and demand data. Therefore, the full CNY 165,052 (12.18%) reduction in total cost is attributable entirely to enabling hinterland lateral repositioning.

Decomposing the marginal effect (Table 4): enabling lateral land routes introduces a new cost of CNY 110,718 (hinterland lateral repositioning) but avoids CNY 269,958 in sea–land repositioning, CNY 10,500 in leasing, and CNY 2,032 in seaport feeder repositioning, yielding a net saving of CNY 165,052 after accounting for a CNY 6,720 increase in holding cost (the latter reflects longer dwell times caused by the expanded set of inland storage nodes). The decomposition confirms that the primary efficiency gain of lateral land routes is their substitution effect on costlier sea–dry corridors.

We have added a clarifying sentence in Section 5.3 explicitly framing the 12.18% reduction as the marginal effect of the lateral land route under the controlled scenario pair.

3.14.3 Modifications

1. **Section 5.3 (Results Analysis) — ADD clarifying sentence.**

   After: "...enabling hinterland lateral repositioning yields a 12.18% reduction in total operational costs."

   Added: "Because the two scenarios differ solely in whether hinterland lateral repositioning is enabled, this entire reduction is the marginal effect attributable to the lateral land route: the CNY 110,718 spent on hinterland lateral repositioning avoids CNY 269,958 in sea–land repositioning, CNY 10,500 in leasing, and CNY 2,032 in seaport feeder repositioning, yielding a net saving of CNY 165,052 after accounting for a CNY 6,720 increase in holding cost."

3.15 Comment 4.29 — Human Expert as Core Architectural Layer

3.15.1 Comment

"The conclusion relies on human expert integration as a caveat rather than synthesizing it as a core architectural layer of the proposed system."

3.15.2 Response

We are grateful to the reviewer for this valuable suggestion; positioning the human expert as a core architectural layer rather than a caveat is more faithful to our design and a stronger way to frame the work. We have revised the conclusion to reposition human-in-the-loop integration as a deliberate architectural design rather than a limitation, stating that domain experts retain ultimate responsibility for model validity, constraint fidelity, and operational feasibility. This reframing reflects our design philosophy that high-stakes optimization requires both mathematical rigor and domain-specific judgment.

3.15.3 Modifications

1. **Section 6 (Conclusions) — MODIFY human expert framing**

   Before: "At present, LLM-based AI agents are incapable of fully supplanting human experts; domain-specific agents primarily serve as auxiliary AI copilots. Consequently, the proposed method has been experimentally validated solely within the context of MIP and cannot entirely substitute human expertise."

   After: "SOP-MAS is deliberately designed as an expert-in-the-loop decision support system, where human expertise constitutes a core architectural layer rather than a residual limitation. Domain experts retain ultimate responsibility for model validity, constraint fidelity, and operational feasibility."